Aplicações e Serviços na Web 2025/2026 - CHECKPOINT 2

RunX

Plataforma Web de Gestão Diária de Corridas

DOCUMENTO
Evolução para:
Checkpoint 2

GRUPO
Grupo 16

ENTREGA
Abril
de 2026

ELEMENTOS

José Resende, 62513
Miguel Pereira, 64146
Nelinton Almeida, 60931
Rodrigo Malato, 64129

Faculdade de Ciências da Universidade de Lisboa

Conteúdo

1 Enquadramento **2**
1.1 Apresentação do Projeto . . . 2
1.2 Problema a Resolver . . . 2
1.3 Tópicos Escolhidos e Motivação . . . 2

2 Utilizadores e Cenários **3**
2.1 Tipos de Utilizadores . . . 3
2.2 Cenários de Utilização Principais . . . 3
2.2.1 Cenário 1 – Registo diário de corrida (Corredor) . . . 3
2.2.2 Cenário 2 – Organização de corrida coletiva (Organizador) . . . 3
2.2.3 Cenário 3 – Exploração do histórico e sugestão de trajeto (Corredor) . . . 4
2.2.4 Cenário 4 – Análise semanal e relatório com LLM (Corredor) . . . 4

3 Storyboards e Sketches de Interação **5**

4 Casos de Uso **12**
4.1 UC-01: Registar Corrida . . . 12
4.2 UC-02: Consultar Histórico de Corridas . . . 13
4.3 UC-03: Criar Evento Coletivo . . . 13
4.4 UC-04: Gerir Membros do Grupo . . . 14
4.5 UC-05: Obter Sugestão de Trajeto via LLM . . . 14
4.6 UC-06: Editar Perfil e Configurações . . . 15

5 Sistema e Funcionalidades **16**
5.1 Recursos do Sistema . . . 16
5.2 Funcionalidades Principais . . . 16
5.2.1 Gestão de Corridas . . . 16
5.2.2 Progresso e Metas . . . 16
5.2.3 Social e Colaboração . . . 17
5.2.4 Contexto e Notificações . . . 17
5.2.5 Perfil e Conta . . . 17
5.2.6 Funcionalidades com LLM . . . 17
5.3 Fluxos Principais do Sistema . . . 18

6 Requisitos Funcionais e Não Funcionais **19**
6.1 Requisitos Gerais do Sistema . . . 19
6.2 Requisitos por Fase . . . 20
6.2.1 F1 – Especificação (18 Mar) . . . 20
6.2.2 F2 – Backend e Serviços . . . 20
6.2.3 F3 – Integração e Frontend . . . 20
6.2.4 F4 – Versão Final . . . 20
6.3 Requisitos Não Funcionais . . . 21

7 Arquitetura Inicial e Modelo E-A **22**
7.1 Visão Geral da Arquitetura . . . 22
7.2 Organização do Backend . . . 22
7.3 Diagrama Entidade-Associação . . . 22
7.4 Exemplos de Endpoints da API . . . 23
7.4.1 Autenticação . . . 23
7.4.2 Utilizadores e Perfil . . . 24
7.4.3 Corridas (Runs) . . . 24
7.4.4 Grupos e Eventos Coletivos . . . 24
7.4.5 Feed Social . . . 24
7.4.6 Funcionalidades LLM . . . 25

8 Fluxos de Integração **26**
8.1 Fluxo: Registar Corrida com Dados Meteorológicos 26
8.2 Fluxo: Sugestão de Variante de Trajeto via LLM 27

9 Serviços Externos e LLM **28**
9.1 Serviços Externos 28
9.1.1 Google Maps / Mapbox – Mapeamento e GPS 28
9.1.2 Cloudinary / AWS S3 – Armazenamento de Imagens 28
9.1.3 Firebase Cloud Messaging – Notificações Push 28
9.1.4 OpenWeather API – Dados Meteorológicos 28
9.2 Funcionalidades com LLM 28
9.2.1 Sugestão Automática de Trajetos 28
9.2.2 Geração de Frases Motivacionais 29
9.2.3 Análise de Padrões e Relatório Semanal 29

10 Decisões de Projeto **30**
10.1 D-01: Mix de dois tópicos do enunciado 30
10.2 D-02: Dois tipos de utilizador 30
10.3 D-03: PostgreSQL como base de dados 30
10.4 D-04: JWT para autenticação 31
10.5 D-05: LLM via API externa 31
10.6 D-06: Publicações anonimizadas 31
10.7 D-07: Storyboards como documento HTML 32

11 Evolução do Documento para o Checkpoint 2 **33**
11.1 Objetivo da evolução 33
11.2 Reflexão por secção do documento base 33
11.2.1 Enquadramento 33
11.2.2 Utilizadores e Cenários 33
11.2.3 Storyboards e Interação 33
11.2.4 Casos de Uso 33
11.2.5 Requisitos Funcionais 33
11.2.6 Requisitos Não Funcionais 34
11.2.7 Modelo de Dados e API 34
11.3 Alterações implementadas no Checkpoint 2 34
11.4 Anexo A – Estimativa de cumprimento de requisitos 34
11.4.1 A.1 Requisitos gerais 34
11.4.2 A.2 Requisitos específicos por fase 35
11.5 Riscos remanescentes e plano imediato 35

Resumo Executivo **36**

1

Enquadramento

1.1 Apresentação do Projeto

O **RunX** é uma plataforma Web de gestão diária de corridas que combina o registo individual de treinos com dinâmicas sociais e colaborativas. A aplicação permite que os utilizadores registem as suas corridas, acompanhem o seu progresso semanal e partilhem experiências com outros corredores – individualmente ou em grupo.

A plataforma integra informação contextual (localização, tempo, clima, rotinas) com funcionalidades de arquivo vivo, permitindo que cada corrida se torne não apenas um registo de desempenho, mas também uma memória narrativa associada a um lugar, um estado de espírito ou um tema.

1.2 Problema a Resolver

As aplicações de corrida existentes focam-se maioritariamente em métricas de desempenho (distância, ritmo, calorias), ignorando a dimensão narrativa e social do ato de correr. Por outro lado, as plataformas sociais de fitness carecem de ferramentas de organização pessoal e contextualização temporal.

O RunX resolve esta lacuna ao:

- Combinar registo de desempenho com memória narrativa (foto, observações, frase motivacional)
- Integrar lembretes contextuais com base em rotinas, localização e progresso semanal
- Permitir corridas coletivas com partilha de trajetos e publicações conjuntas
- Sugerir trajetos e analisar padrões usando inteligência artificial

1.3 Tópicos Escolhidos e Motivação

Mix de dois tópicos do enunciado
Gestão de Tarefas Contextual e Social – o ato de correr é tratado como uma tarefa recorrente, com contexto (tempo, localização, rotina, estado) e dimensão social (grupos, partilha, corridas coletivas, sugestões entre utilizadores).
Life Threads: Diário Digital Colaborativo – cada corrida torna-se um registo no arquivo vivo do utilizador, com foto, observações, tema e frase motivacional, pesquisável e partilhável, construindo uma narrativa evolutiva de experiências pessoais e coletivas ao longo do tempo.

Esta combinação é natural: correr é simultaneamente um hábito a gerir e uma experiência a recordar. O RunX explora essa dualidade para criar uma plataforma coerente que vai além do simples tracking de atividade física.

2

Utilizadores e Cenários

2.1 Tipos de Utilizadores

O sistema define dois perfis principais com permissões distintas:

Perfil	Corredor (Runner)	Organizador de Grupo
Descrição	Utilizador padrão da plataforma. Regista corridas, acompanha o progresso semanal, partilha publicações e participa em eventos coletivos.	Utilizador com permissões elevadas para gerir grupos. Cria eventos coletivos, modera publicações do grupo e convida membros.
Permissões	CRUD sobre as suas próprias corridas; leitura do feed social; participação em grupos; gestão do seu perfil.	Todas as permissões do Corredor, mais: criar/editar/eliminar grupos; criar eventos coletivos; gerir membros; moderar conteúdo do grupo.
Acesso API	`/runs`, `/profile`, `/feed`, `/groups` (leitura e participação)	Endpoints adicionais: `/groups` (escrita), `/groups/:id/events`, `/groups/:id/members`

Tabela 2.1: Tipos de utilizadores e suas permissões

2.2 Cenários de Utilização Principais

2.2.1 Cenário 1 – Registo diário de corrida (Corredor)

A Sofia é corredora habitual e usa o RunX para registar os seus treinos diários. Na sexta-feira ao fim da tarde, recebe uma notificação push do RunX a lembrar-lhe que faltam 5.8 km para atingir a meta semanal de 40 km. A Sofia sai para correr pela Ribeira de Lisboa, com o GPS ativo na app.

No final da corrida, abre o RunX e vê as métricas registadas automaticamente (6.2 km, 52 min, ritmo de 8:23/km). Tira uma fotografia ao pôr do sol sobre o Tejo, seleciona o tema *Amanhecer*, escreve uma frase motivacional e partilha a publicação com o seu grupo *Ribeira Runners*. O dashboard atualiza-se imediatamente, mostrando a meta semanal atingida.

2.2.2 Cenário 2 – Organização de corrida coletiva (Organizador)

O Pedro é organizador do grupo *Monsanto Trail Crew*. Decide criar um evento coletivo para o sábado seguinte: uma corrida de 10 km no Parque de Monsanto com o tema *Natureza*. Na página de gestão do grupo, cria o evento, define o trajeto no mapa e envia convites aos 12 membros do grupo.

Os membros recebem uma notificação push com o convite. Oito confirmam presença. No dia do evento, cada corredor regista a sua corrida individualmente, mas todas ficam associadas ao evento coletivo, gerando uma publicação conjunta no feed.

2.2.3 Cenário 3 – Exploração do histórico e sugestão de trajeto (Corredor)

O João quer explorar o seu arquivo de corridas. Na secção Histórico, pesquisa por *'Monsanto'* e filtra pelo tema *Natureza*. A app mostra-lhe todas as corridas naquele parque, com os mapas dos trajetos e as notas que foi escrevendo. Ao abrir uma corrida, a funcionalidade de LLM sugere-lhe uma variante do trajeto com 2 km adicionais, baseada nos seus padrões de corrida e preferências do perfil.

2.2.4 Cenário 4 – Análise semanal e relatório com LLM (Corredor)

No final de cada semana, o RunX gera automaticamente um relatório personalizado com análise dos padrões de corrida: dias mais produtivos, ritmo médio por tema, evolução mensal e uma sugestão de meta para a semana seguinte. O relatório inclui também uma frase motivacional gerada por LLM com base nas corridas registadas durante a semana.

3

Storyboards e Sketches de Interação

Os storyboards foram desenvolvidos em papel , cobrindo **12 ecrãs** organizados em **6 fluxos principais**. As imagens que digitalizámos dos nossos sketches foram anexadas a este documento.

Fluxo	Nome	Ecrãs incluídos
Fluxo 01	Entrada & Dashboard Semanal	Login, Dashboard com strip semanal, Notificações push contextuais
Fluxo 02	Registar & Documentar uma Corrida	Dados GPS + mapa, Foto + memória (tema, frase), Publicação social, Confirmação
Fluxo 03	Histórico, Pesquisa & Exploração	Timeline de corridas com filtros por tema, Detalhe completo de corrida
Fluxo 04	Feed Social & Corridas Coletivas	Feed agregado com publicações, Evento coletivo (criação e participação)
Fluxo 05	Diagrama de Navegação	Visão geral das transições entre todos os ecrãs
Fluxo 06	Perfil do Utilizador	Vista do perfil com estatísticas, Editar perfil (avatar, username, password), Definições e notificações

Tabela 3.1: Fluxos de storyboards

Nota sobre os storyboards
As imagens que apresentamos de seguida contéem sketches interativos de alta fidelidade para todos os ecrãs descritos acima, com anotações detalhadas por ecrã e diagrama de navegação completo.

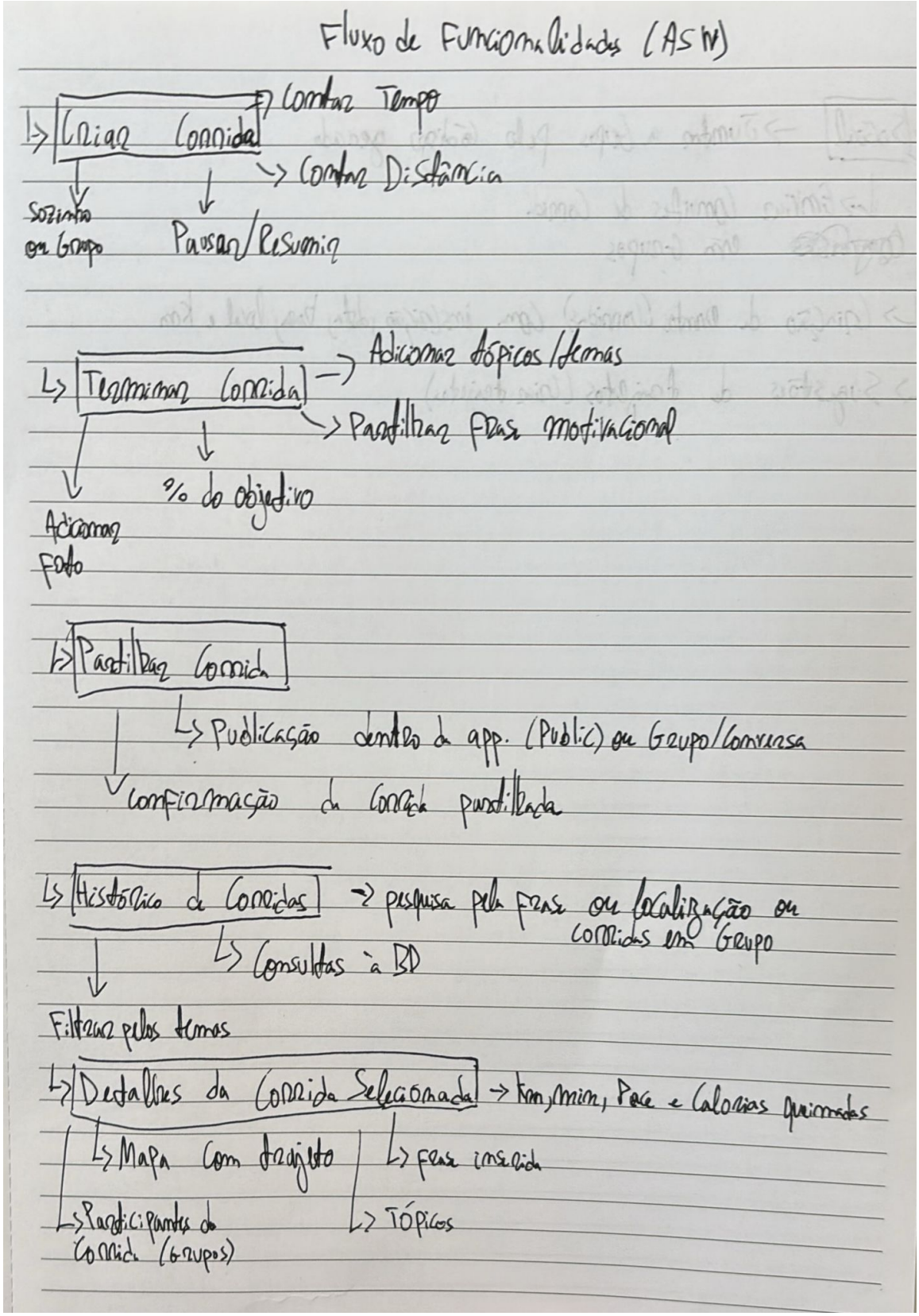

Figura 3.1: Fluxo 1

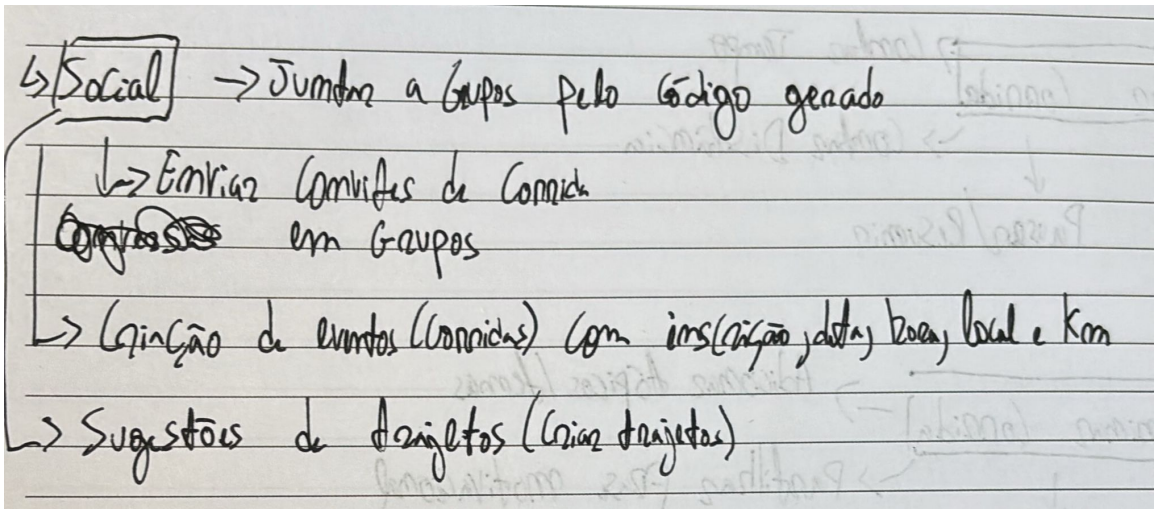

Figura 3.2: Fluxo 2

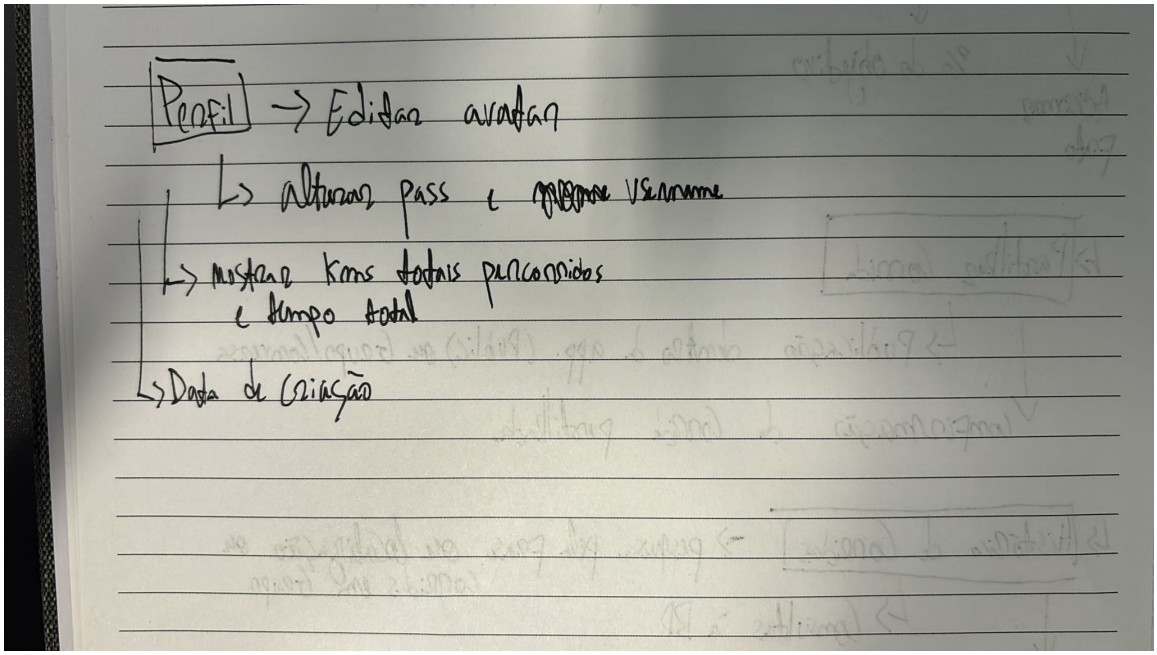

Figura 3.3: Fluxo 3

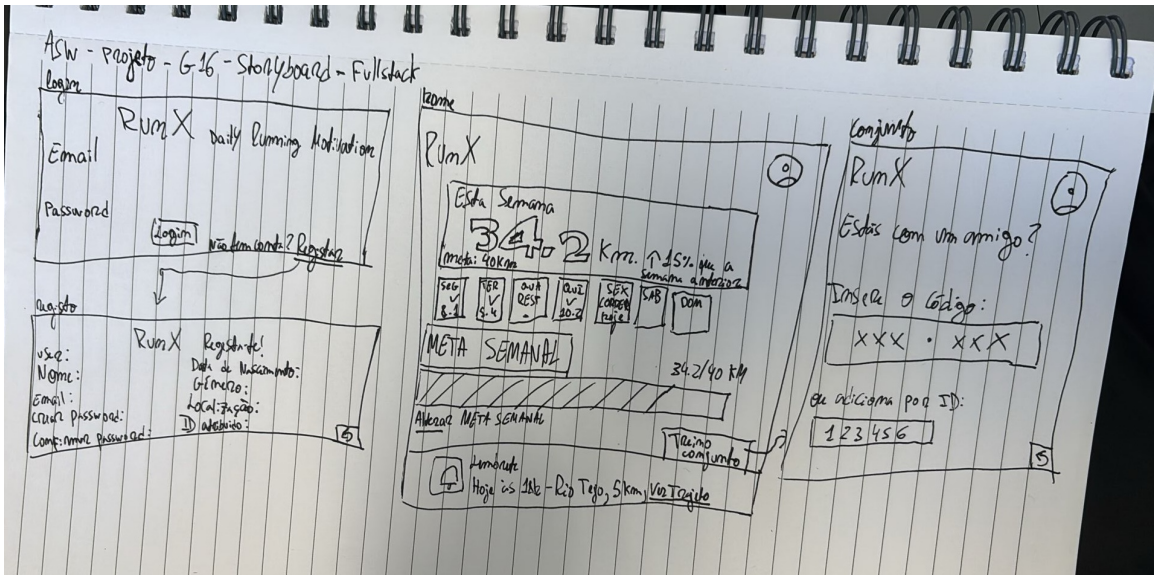

Figura 3.4: Storyboard 1

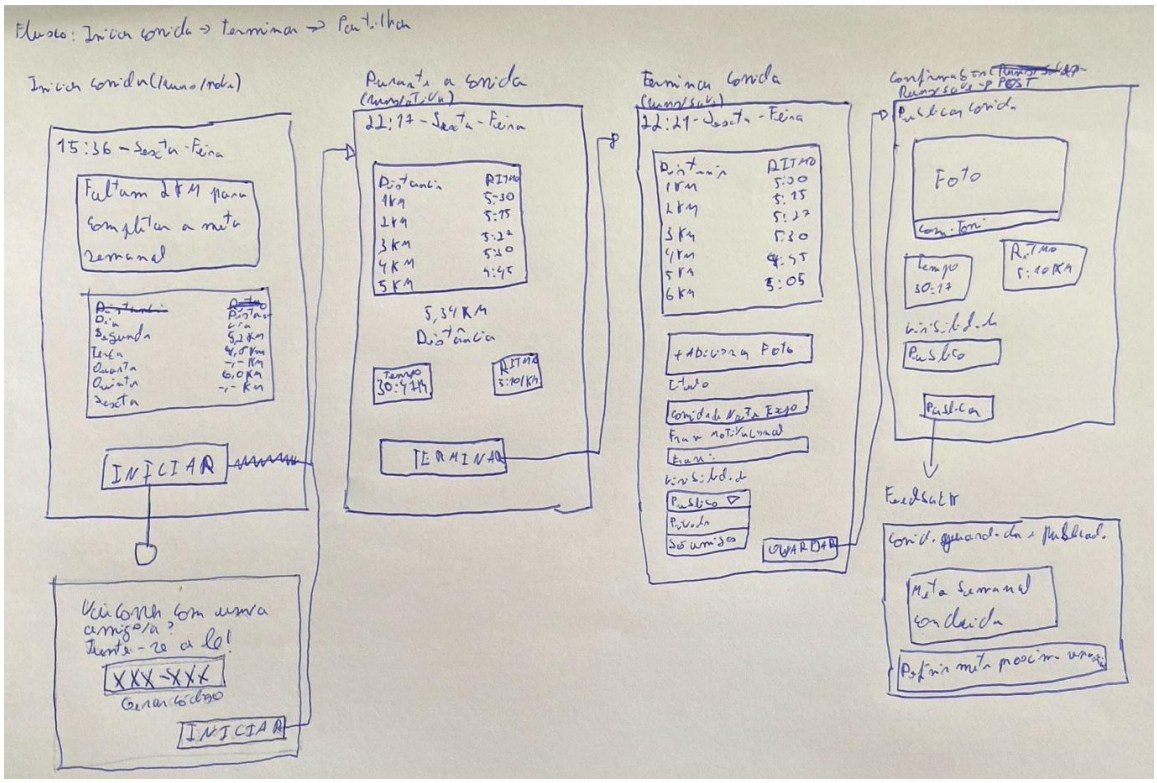

Figura 3.5: Storyboard 2

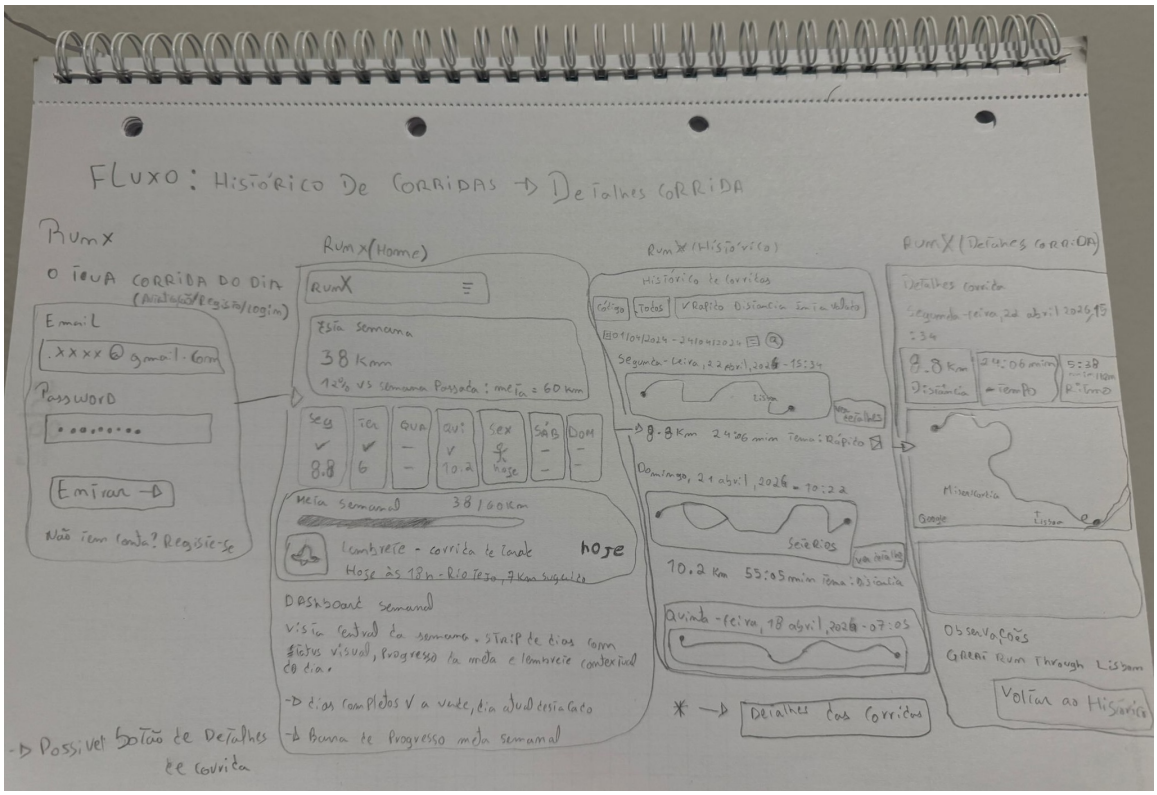

Figura 3.6: Storyboard 3

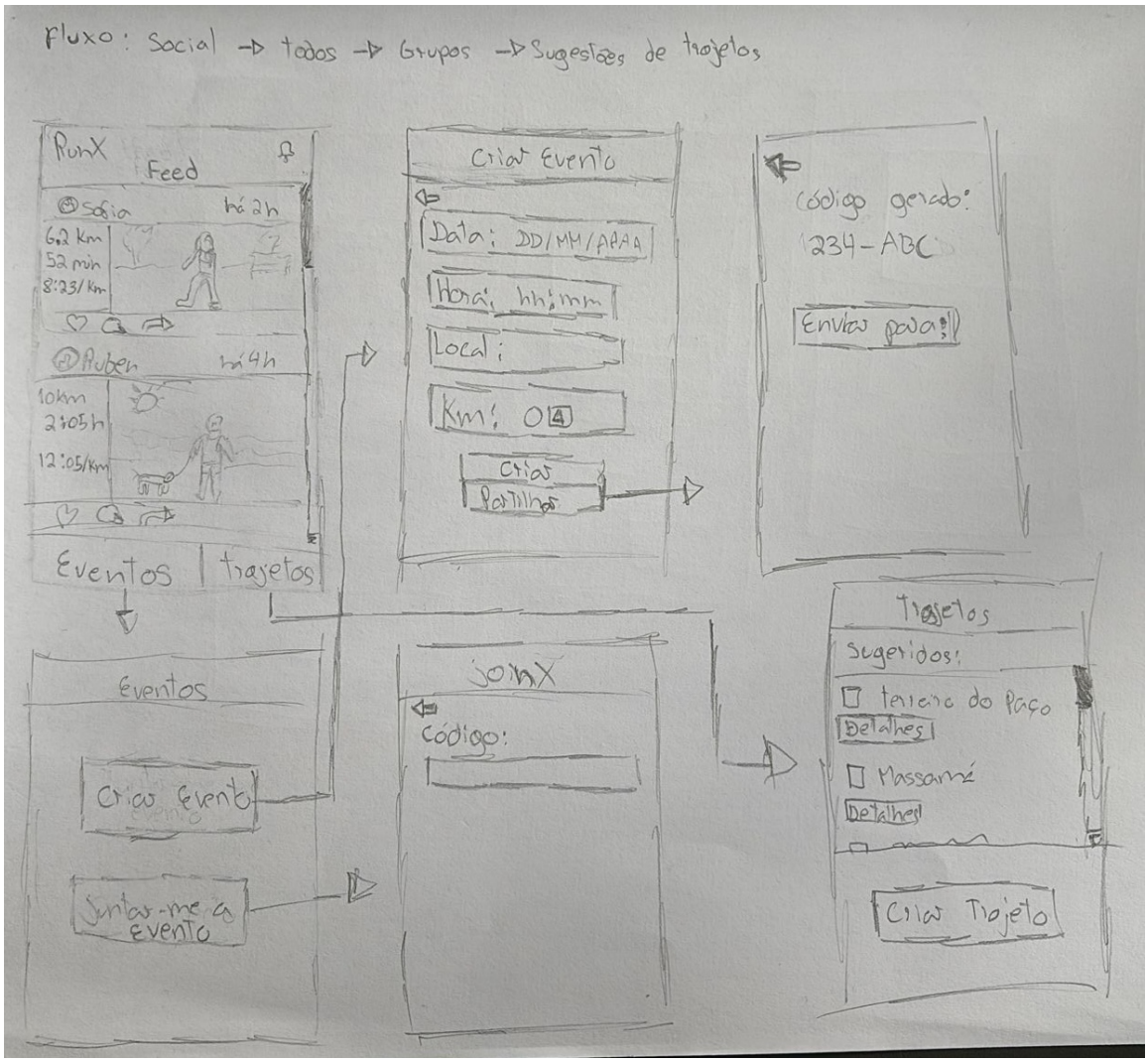

Figura 3.7: Storyboard 4

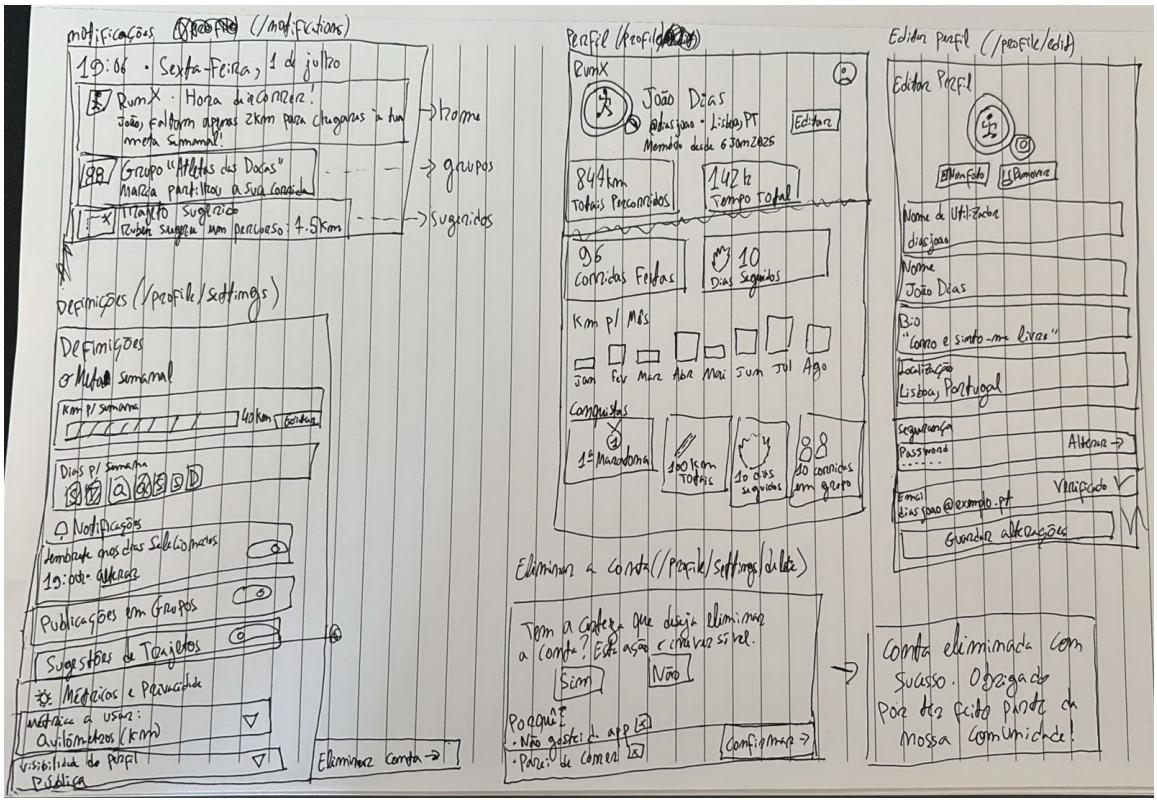

Figura 3.8: Storyboard 5

4

Casos de Uso

Os casos de uso descrevem as interações principais entre os utilizadores e o sistema, com pré-condições e pós-condições explícitas conforme o modelo UP (Unified Process).

4.1 UC-01: Registar Corrida

UC-01 Registar Corrida Actor: Corredor

Pré-condições	Utilizador autenticado; GPS ativo no dispositivo.
Pós-condições	Corrida guardada na BD com métricas, trajeto, foto e tema; progresso semanal atualizado; publicação criada (se visibilidade $\neq$ privado).
Fluxo Principal	1. Utilizador inicia o registo de corrida na app. 2. Sistema começa a recolher dados GPS (distância, ritmo, tempo). 3. Utilizador termina a corrida. 4. Sistema calcula métricas finais e apresenta o formulário de memória. 5. Utilizador faz upload de foto, escreve observações e seleciona tema. 6. LLM sugere frase motivacional (pré-preenchida, editável). 7. Utilizador define visibilidade e confirma publicação. 8. Sistema guarda a corrida, atualiza o dashboard semanal e cria o post.
Alternativo	Falha de GPS: sistema permite inserção manual de distância e tempo.

4.2 UC-02: Consultar Histórico de Corridas

UC-02 Consultar Histórico de Corridas Actor: Corredor

Pré-condições	Utilizador autenticado; pelo menos uma corrida registada.
Pós-condições	Lista de corridas apresentada com filtros aplicados; detalhe de corrida acessível.
Fluxo Principal	1. Utilizador navega para a secção Histórico. 2. Sistema apresenta timeline de corridas ordenada por data. 3. Utilizador aplica filtros (tema, localização, intervalo de datas, texto livre). 4. Sistema filtra e apresenta os resultados correspondentes. 5. Utilizador seleciona uma corrida para ver o detalhe. 6. Sistema apresenta métricas, mapa do trajeto, foto e observações.
Alternativo	Sem resultados para os filtros aplicados: sistema apresenta mensagem informativa.

4.3 UC-03: Criar Evento Coletivo

UC-03 Criar Evento Coletivo Actor: Organizador de Grupo

Pré-condições	Utilizador autenticado com `role=organizer`; grupo existente com pelo menos 2 membros.
Pós-condições	Evento criado e guardado na BD; notificação push enviada a todos os membros do grupo.
Fluxo Principal	1. Organizador acede à página de gestão do grupo. 2. Organizador clica em 'Criar Evento' e preenche: título, data/hora, trajeto no mapa, tema. 3. Sistema valida os dados e cria o evento. 4. Sistema envia notificação push (via FCM) a todos os membros. 5. Membros podem confirmar ou recusar presença.
Alternativo	Erro no envio de notificações: evento é criado, notificações são colocadas em fila para reenvio.

4.4 UC-04: Gerir Membros do Grupo

UC-04 Gerir Membros do Grupo Actor: Organizador de Grupo

Pré-condições Utilizador autenticado com `role=organizer` no grupo em questão.

Pós-condições Membro adicionado ou removido do grupo; lista de membros atualizada.

Fluxo Principal

1. Organizador acede à página de gestão de membros do grupo.
2. Para adicionar: pesquisa utilizador por username e envia convite.
3. Utilizador convidado recebe notificação e aceita/recusa.
4. Para remover: organizador seleciona membro e confirma remoção.
5. Sistema atualiza a tabela `group_members`.

Alternativo Utilizador a remover é o único organizador: sistema bloqueia a ação e apresenta aviso.

4.5 UC-05: Obter Sugestão de Trajeto via LLM

UC-05 Obter Sugestão de Trajeto via LLM Actor: Corredor

Pré-condições Utilizador autenticado; corrida com trajeto GPS registado.

Pós-condições Sugestão de variante de trajeto gerada e apresentada; guardada na BD associada à corrida.

Fluxo Principal

1. Utilizador abre o detalhe de uma corrida e clica em 'Sugerir variante'.
2. Sistema envia ao LLM: trajeto atual, histórico de corridas na área, preferências do utilizador.
3. LLM devolve descrição textual da variante com pontos de interesse e distância estimada.
4. Sistema guarda a sugestão e apresenta-a ao utilizador no mapa.

Alternativo Falha da API LLM: sistema apresenta mensagem de indisponibilidade temporária.

4.6 UC-06: Editar Perfil e Configurações

UC-06 Editar Perfil e Configurações Actor: Corredor / Organizador

Pré-condições Utilizador autenticado.

Pós-condições Dados do perfil atualizados na BD; novo avatar carregado para o serviço de armazenamento (se alterado).

Fluxo Principal

1. Utilizador acede à página de perfil e clica em 'Editar'.
2. Sistema apresenta formulário com dados atuais.
3. Utilizador altera campos desejados (username, bio, avatar, localização).
4. Para alterar password: utilizador introduz password atual e nova password ($\times 2$).
5. Sistema valida e guarda as alterações.
6. Se avatar alterado: imagem enviada para Cloudinary/S3, URL atualizado na BD.

Alternativo Username já existente: sistema apresenta erro de validação sem guardar.

5

Sistema e Funcionalidades

5.1 Recursos do Sistema

Recurso	Descrição
User	Conta de utilizador com perfil, preferências e permissões (runner / organizer)
Run	Registo de uma corrida: métricas GPS, trajeto, foto, observações, tema, frase motivacional
Group	Grupo de corredores com membros e organizador(es)
GroupEvent	Evento coletivo criado por um organizador, associado a um grupo
Post	Publicação social derivada de um Run ou GroupEvent, visível no feed
Theme	Etiqueta temática associada a uma corrida (ex: Natureza, Urbano, Amanhecer)
Notification	Lembrete ou alerta gerado pelo sistema ou por atividade social
WeeklyGoal	Meta semanal de km definida pelo utilizador, com progresso calculado
RouteSuggestion	Sugestão de variante de trajeto gerada por LLM
WeeklyReport (virtual)	Relatório semanal gerado por LLM com análise de padrões (não persistido como entidade própria nesta fase)

Tabela 5.1: Recursos principais do sistema

5.2 Funcionalidades Principais

5.2.1 Gestão de Corridas

- Registo de corrida com métricas automáticas (distância, tempo, ritmo, calorias) via GPS
- Upload de fotografia da corrida
- Adição de observações livres, frase motivacional e tema
- Consulta e pesquisa do histórico de corridas (por data, localização, tema, texto livre)
- Visualização de trajeto GPS em mapa interativo

5.2.2 Progresso e Metas

- Dashboard semanal com strip de dias, barra de progresso e meta configurável
- Gráfico de km mensais no perfil
- Conquistas (badges) desbloqueadas por milestones
- Sequência de dias consecutivos com corrida registada (streak)

5.2.3 Social e Colaboração

- Feed social com publicações de corridas da rede do utilizador
- Reações e comentários em publicações
- Grupos de corrida com gestão de membros pelo organizador
- Criação de eventos coletivos com trajeto, data/hora e tema partilhado
- Publicação conjunta automática após evento coletivo
- Reutilização de trajetos de outros utilizadores ("Usar trajeto")

5.2.4 Contexto e Notificações

- Lembretes diários configuráveis por hora e dias da semana
- Notificações push contextuais (progresso, atividade social, sugestões de trajeto)
- Consulta de condições meteorológicas em tempo real para enriquecer a experiência de corrida (OpenWeather)

5.2.5 Perfil e Conta

- Edição de avatar, username, nome, bio e localização
- Alteração de password e email
- Estatísticas totais: km, horas, nº de corridas, data de criação da conta
- Configuração de unidades (km/milhas) e visibilidade do perfil

5.2.6 Funcionalidades com LLM

- Sugestão automática de variantes de trajeto baseada em histórico e preferências
- Geração de frases motivacionais personalizadas no registo de corrida
- Análise de padrões e geração de relatório semanal personalizado

5.3 Fluxos Principais do Sistema

Fluxo	Sequência
Registo de corrida	Autenticação -> Dashboard -> Nova Corrida (GPS) -> Foto + Memória -> Publicar/Guardar -> Dashboard atualizado
Descoberta e exploração	Dashboard -> Histórico -> Pesquisa/Filtro -> Detalhe de Corrida -> Sugestão de variante (LLM)
Corrida coletiva	Organizador cria evento -> Membros recebem convite -> Confirmação de presença -> Corrida individual -> Publicação conjunta gerada
Relatório semanal	Fim de semana -> LLM analisa corridas -> Gera relatório + frase motivacional -> Notificação push enviada
Gestão de grupo	Organizador cria grupo -> Convida membros -> Define evento -> Gere membros e modera conteúdo

Tabela 5.2: Fluxos principais do sistema

6

Requisitos Funcionais e Não Funcionais

6.1 Requisitos Gerais do Sistema

ID	Requisito	Fase	Tipo
RG-01	Plataforma multiutilizador com autenticação por JWT	F1	Plataforma
RG-02	Dois tipos de utilizador: Corredor e Organizador de Grupo	F1	Plataforma
RG-03	Fluxos com interação entre utilizadores (feed, grupos, eventos)	F1	Plataforma
RG-04	Backend implementado em Node.js com Express	F2	Backend
RG-05	Persistência de dados com PostgreSQL	F2	Backend
RG-06	Modelo de dados coerente com os recursos definidos	F2	Backend
RG-07	Operações CRUD sobre todos os recursos principais	F2	Backend
RG-08	API organizada em camadas: routing, lógica de negócio, acesso a dados	F2	API
RG-09	Documentação da API com OpenAPI/Swagger	F2	API
RG-10	Frontend implementado em React, integrado com a API	F3	Frontend
RG-11	Integração com pelo menos um serviço externo	F3	Integrações
RG-12	Integração com funcionalidade baseada em LLM	F3	Integrações
RG-13	Autenticação e controlo de acesso a todos os recursos protegidos	F2	Segurança
RG-14	Validação de dados de entrada em todos os endpoints	F2	Segurança
RG-15	Tratamento consistente de erros com respostas HTTP adequadas	F2	Segurança
RG-16	Configuração via variáveis de ambiente (.env)	F2	Segurança

Tabela 6.1: Requisitos gerais do sistema

6.2 Requisitos por Fase

6.2.1 F1 – Especificação (18 Mar)

ID	Requisito	Tipo
RF-01	Documento de especificação com enquadramento, utilizadores e cenários	Funcional
RF-02	Storyboards e sketches de interação para todos os fluxos principais	Funcional
RF-03	Lista de funcionalidades, recursos e requisitos definidos	Funcional
RF-04	Modelo de dados preliminar e exemplos de endpoints da API	Técnico
RF-05	Identificação de serviços externos e ideias para LLM	Técnico

Tabela 6.2: Requisitos da Fase 1

6.2.2 F2 – Backend e Serviços

ID	Requisito	Tipo
RF-06	API REST completa com autenticação JWT e controlo de acesso	Funcional
RF-07	Endpoints CRUD para: Users, Runs, Groups, GroupEvents, Posts	Funcional
RF-08	Persistência em PostgreSQL com migrações e modelo relacional	Técnico
RF-09	Validação de inputs e tratamento de erros em todos os endpoints	Técnico
RF-10	Documentação OpenAPI/Swagger disponível em `/api-docs`	Técnico
RF-11	Integração inicial com OpenWeather API (clima na corrida)	Integração

Tabela 6.3: Requisitos da Fase 2

6.2.3 F3 – Integração e Frontend

ID	Requisito	Tipo
RF-12	Frontend React com todos os fluxos principais implementados	Funcional
RF-13	Dashboard semanal, registo de corrida, histórico e feed social	Funcional
RF-14	Integração com Mapbox/Google Maps para mapas e trajetos GPS	Integração
RF-15	Integração com Cloudinary/S3 para upload de imagens	Integração
RF-16	Integração com Firebase Cloud Messaging para notificações push	Integração
RF-17	Integração com LLM para sugestão de trajetos e frases motivacionais	LLM

Tabela 6.4: Requisitos da Fase 3

6.2.4 F4 – Versão Final

ID	Requisito	Tipo
RF-18	Relatório semanal personalizado gerado por LLM	LLM
RF-19	Sistema de conquistas (badges) e sequências de dias	Funcional
RF-20	Testes de integração nos fluxos principais	Técnico
RF-21	Documentação final e discussão de decisões arquiteturais	Técnico

Tabela 6.5: Requisitos da Fase 4

6.3 Requisitos Não Funcionais

ID	Categoria	Requisito
RNF-01	Desempenho	A API deve responder a pedidos simples em menos de 300ms em condições normais de carga.
RNF-02	Desempenho	O sistema deve suportar pelo menos 50 utilizadores simultâneos sem degradação visível.
RNF-03	Segurança	Todas as passwords são armazenadas com hash bcrypt (fator de custo $\geq$ 10). Nunca em texto simples.
RNF-04	Segurança	Tokens JWT expiram ao fim de 1 hora; refresh tokens ao fim de 7 dias. Comunicação apenas por HTTPS.
RNF-05	Segurança	Todas as operações de escrita validam que o utilizador autenticado tem permissão sobre o recurso.
RNF-06	Disponibilidade	O sistema deve ter disponibilidade de 99% durante o período de avaliação do projeto.
RNF-07	Usabilidade	A interface deve ser responsiva e funcional em browsers modernos (Chrome, Firefox, Safari).
RNF-08	Manutenibilidade	O código deve seguir uma estrutura em camadas clara (routes / controllers / services / models).
RNF-09	Manutenibilidade	Toda a configuração sensível deve ser gerida por variáveis de ambiente (.env).
RNF-10	Portabilidade	A aplicação deve poder ser executada localmente com um único comando.
RNF-11	Resiliência	Falhas em serviços externos não devem bloquear o fluxo principal.
RNF-12	Dados (RGPD)	O sistema deve suportar eliminação de conta com remoção de dados pessoais.

Tabela 6.6: Requisitos não funcionais

7

Arquitetura Inicial e Modelo E-A

7.1 Visão Geral da Arquitetura

O RunX segue uma arquitetura cliente-servidor em três camadas, com uma API REST no centro do sistema:

- **Frontend (React)** – SPA servida ao cliente, consome a API via HTTP
- **Backend (Node.js + Express)** – API REST organizada em rotas, controladores e serviços
- **Persistência (PostgreSQL)** – base de dados relacional com modelo normalizado
- **Serviços Externos** – integrados no backend, não expostos diretamente ao frontend

7.2 Organização do Backend

Pasta	Responsabilidade
`routes/`	Definição dos endpoints e middlewares de autenticação/autorização
`controllers/`	Lógica de tratamento de pedidos e respostas HTTP
`services/`	Lógica de negócio: validações, regras, cálculos
`models/`	Queries à base de dados (ou ORM) e definições de tipos
`integrations/`	Clientes para serviços externos (Maps, Weather, LLM, Storage, FCM)
`middlewares/`	Autenticação JWT, controlo de acesso, validação, erros
`config/`	Variáveis de ambiente, configuração de base de dados e serviços

Tabela 7.1: Organização do backend

7.3 Diagrama Entidade-Associação

Diagrama E-A preliminar do RunX com 13 entidades, atributos principais e relações com cardinalidades. As entidades estão organizadas em 4 colunas temáticas: Corridas - Grupos - Social - Auxiliares.

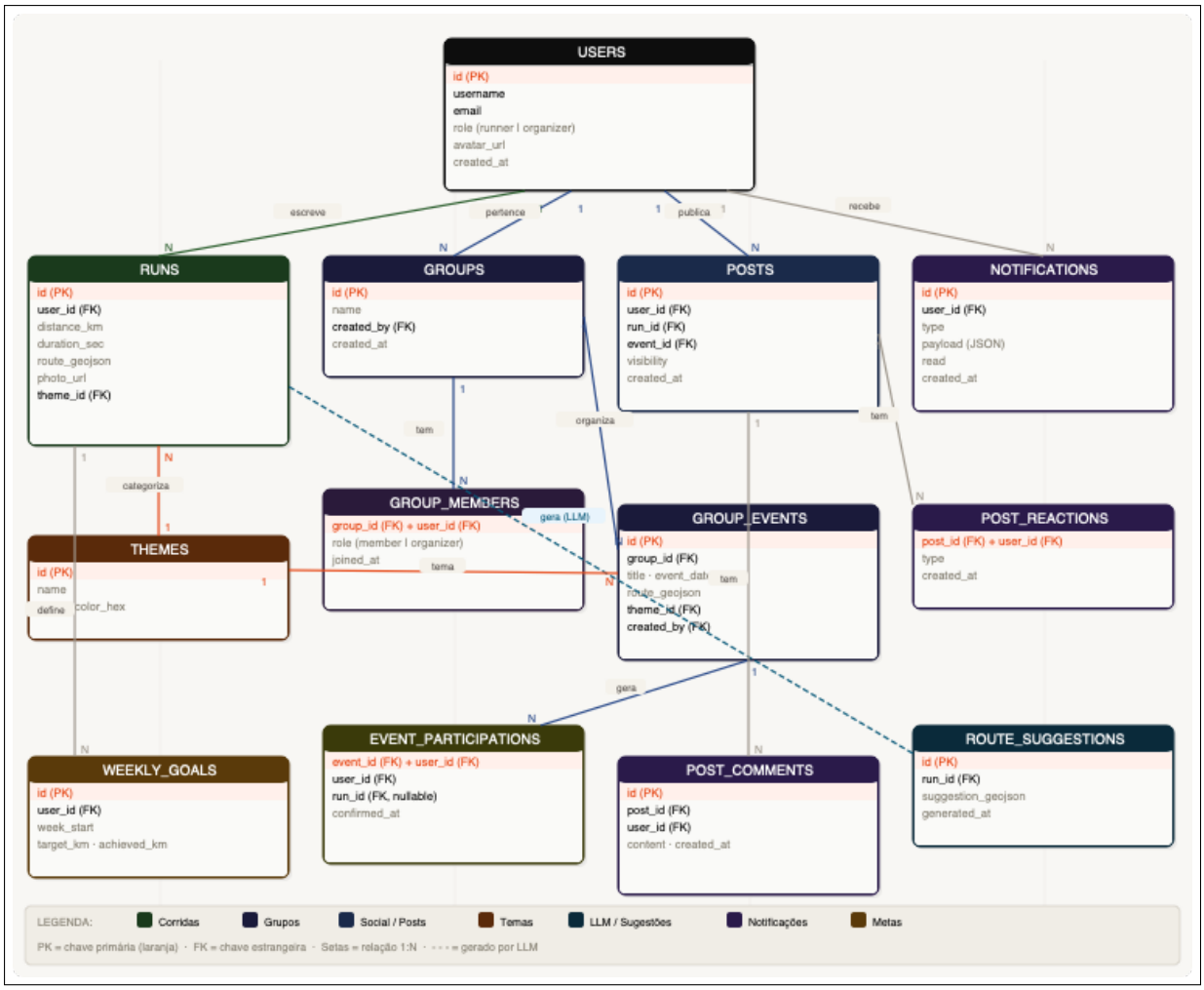

Figura 7.1: Diagrama Entidade-Associação – 13 entidades, relações com cardinalidades

Nota: Um diagrama formal em notação UML/Chen será produzido na Fase 2.

7.4 Exemplos de Endpoints da API

7.4.1 Autenticação

Método	Endpoint	Descrição	Auth
POST	`/auth/register`	Registar novo utilizador	–
POST	`/auth/login`	Autenticar e obter JWT	–
POST	`/auth/logout`	Invalidar token	Auth
POST	`/auth/refresh`	Renovar JWT	Auth

7.4.2 Utilizadores e Perfil

Método	Endpoint	Descrição	Auth
GET	/users/me	Perfil do utilizador autenticado	Auth
PUT	/users/me	Atualizar perfil (nome, bio, localização)	Auth
PUT	/users/me/password	Alterar password	Auth
POST	/users/me/avatar	Upload de avatar (multipart)	Auth
GET	/users/:id	Perfil público de outro utilizador	Auth
GET	/users/me/stats	Estatísticas totais do utilizador	Auth
DELETE	/users/me	Eliminar conta (anonimiza posts)	Auth

7.4.3 Corridas (Runs)

Método	Endpoint	Descrição	Auth
GET	/runs	Listar corridas do utilizador (com filtros)	Auth
POST	/runs	Criar nova corrida	Auth
GET	/runs/:id	Detalhes de uma corrida	Auth
PUT	/runs/:id	Editar corrida (observações, frase, tema)	Auth
DELETE	/runs/:id	Eliminar corrida	Auth
GET	/runs/week	Corridas e progresso da semana atual	Auth
GET	/runs/:id/suggest-route	Sugerir variante de trajeto via LLM	Auth

7.4.4 Grupos e Eventos Coletivos

Método	Endpoint	Descrição	Auth
GET	/groups	Listar grupos do utilizador	Auth
POST	/groups	Criar novo grupo	Auth+Org
GET	/groups/:id	Detalhes de um grupo	Auth
POST	/groups/:id/members	Adicionar membro ao grupo	Auth+Org
DELETE	/groups/:id/members/:uid	Remover membro	Auth+Org
GET	/groups/:id/events	Listar eventos do grupo	Auth
POST	/groups/:id/events	Criar evento coletivo	Auth+Org
POST	/groups/:id/events/:eid/join	Confirmar presença no evento	Auth

7.4.5 Feed Social

Método	Endpoint	Descrição	Auth
GET	/feed	Feed social do utilizador	Auth
POST	/posts/:id/react	Reagir a uma publicação	Auth
POST	/posts/:id/comments	Comentar numa publicação	Auth

7.4.6 Funcionalidades LLM

Método	Endpoint	Descrição	Auth
POST	`/llm/motivational-phrase`	Gerar frase motivacional personalizada	Auth
GET	`/users/me/weekly-report`	Obter relatório semanal gerado por LLM	Auth
POST	`/runs/:id/suggest-route`	Sugestão de variante de trajeto por LLM	Auth

8

Fluxos de Integração com Serviços Externos

Os fluxos seguintes descrevem as sequências de ações internas e externas para as funcionalidades que dependem de serviços externos, ilustrando a complexidade real de integração.

8.1 Fluxo: Registar Corrida com Dados Meteorológicos

Envolve o cliente, o backend RunX, a API OpenWeather e opcionalmente o LLM e o FCM. As linhas destacadas identificam chamadas a serviços externos.

#	Interveniente	Ação / Chamada	Detalhe
1	Cliente -> Backend	`POST /runs`	Envia métricas GPS, coordenadas e timestamp
2	Backend	Validação	Valida campos obrigatórios, verifica JWT
3	Backend -> OpenWeather	`GET /weather`	Pedido de condições meteorológicas
4	OpenWeather -> Backend	`200 OK`	Dados meteorológicos recebidos
5	Backend -> LLM (opcional)	`POST /v1/messages`	Contexto da corrida para frase motivacional
6	LLM -> Backend	`200 OK`	Frase motivacional gerada
7	Backend -> PostgreSQL	`INSERT INTO runs`	Corrida guardada com métricas e metadados principais
8	Backend -> FCM	`POST /fcm/send`	Notificação a seguidores (se post público)
9	Backend -> Cliente	`201 Created`	Resposta com corrida criada

Tabela 8.1: Fluxo de registo de corrida com dados meteorológicos

> **Degradação graciosa:** se OpenWeather falhar (timeout ou erro), a corrida é guardada normalmente e o contexto meteorológico é simplesmente omitido. O utilizador é informado na resposta.

8.2 Fluxo: Sugestão de Variante de Trajeto via LLM

#	Interveniente	Ação / Chamada	Detalhe
1	Cliente -> Backend	`GET /runs/:id/suggest`	Clica em 'Sugerir variante'
2	Backend -> PostgreSQL	`SELECT runs + history`	Corrida atual e últimas 10 na área
3	Backend -> PostgreSQL	`SELECT preferences`	Preferências do utilizador
4	Backend -> LLM API	`POST /v1/messages`	Prompt com trajeto GeoJSON e histórico
5	LLM API -> Backend	`200 OK {JSON}`	Descrição, pontos de interesse, distância
6	Backend	Validação da resposta	Verifica estrutura do JSON
7	Backend -> PostgreSQL	`INSERT INTO route_suggestions`	Guarda sugestão
8	Backend -> Cliente	`200 OK`	Retorna sugestão para o mapa

Tabela 8.2: Fluxo de sugestão de trajeto via LLM

> **Rate limit:** máximo 5 sugestões/utilizador/dia para controlo de custos de tokens.

9

Serviços Externos e LLM

9.1 Serviços Externos

9.1.1 Google Maps / Mapbox – Mapeamento e GPS

Visualização de trajetos GPS no mapa; cálculo de distâncias; geocodificação de localização; sugestão de percursos base. **Integração prevista na Fase 3.**

9.1.2 Cloudinary / AWS S3 – Armazenamento de Imagens

Upload, redimensionamento e entrega otimizada das fotografias de corrida e avatares de perfil. **Integração prevista na Fase 3.**

9.1.3 Firebase Cloud Messaging – Notificações Push

Envio de lembretes diários configuráveis, alertas de atividade social e convites para eventos coletivos. **Integração prevista na Fase 3.**

9.1.4 OpenWeather API – Dados Meteorológicos

Registo automático das condições climatéricas (temperatura, precipitação, vento) no momento e localização de cada corrida. **Integração prevista na Fase 2.**

9.2 Funcionalidades com LLM

O RunX integra um modelo de linguagem (via API da Anthropic ou OpenAI) para três funcionalidades distintas, todas com prompts específicos e respostas estruturadas:

9.2.1 Sugestão Automática de Trajetos

- **Trigger:** utilizador abre o detalhe de uma corrida e clica em 'Sugerir variante'
- **Input ao LLM:** histórico de corridas na mesma área, distância habitual, tema preferido, nível de dificuldade
- **Output esperado:** descrição textual de uma variante do trajeto com pontos de interesse e estimativa de distância
- **Armazenado em:** tabela `route_suggestions`, associada ao `run_id`

9.2.2 Geração de Frases Motivacionais

- **Trigger:** utilizador completa o registo de uma corrida
- **Input ao LLM:** distância, tempo, tema da corrida, histórico recente do utilizador
- **Output esperado:** frase motivacional curta (1-2 frases), personalizada ao contexto
- **Utilização:** pré-preenche o campo de frase motivacional no formulário (editável pelo utilizador)

9.2.3 Análise de Padrões e Relatório Semanal

- **Trigger:** automático ao fim de cada semana (cron job no backend)
- **Input ao LLM:** corridas da semana (distâncias, ritmos, temas, observações), comparação com semanas anteriores
- **Output esperado:** JSON estruturado com análise narrativa, insights de desempenho e sugestão de meta para a semana seguinte
- **Entregue ao utilizador** via notificação push e visível no perfil

> **Considerações de implementação LLM**
>
> - Todos os prompts serão versionados no repositório e configuráveis via variáveis de ambiente.
> - As respostas do LLM serão validadas antes de serem guardadas ou apresentadas ao utilizador.
> - O sistema deve ser resiliente a falhas da API do LLM, apresentando alternativas estáticas quando necessário.
> - O custo de tokens será monitorizado; funcionalidades com LLM terão rate limiting por utilizador.

10

Decisões de Projeto

Esta secção documenta as principais decisões tomadas pelo grupo durante a fase de especificação, com a justificação de cada escolha e as alternativas consideradas.

10.1 D-01: Mix de dois tópicos do enunciado

D-01 Mix de dois tópicos do enunciado

Decisão	Combinar Gestão de Tarefas Contextual e Social com Life Threads: Diário Digital Colaborativo.
Alternativa	Usar apenas um dos tópicos.
Justificação	O ato de correr é simultaneamente uma tarefa a gerir (contexto, lembretes, metas) e uma memória a preservar (foto, frase, tema, arquivo). Os dois tópicos complementam-se naturalmente no domínio escolhido.

10.2 D-02: Dois tipos de utilizador

D-02 Dois tipos de utilizador: Corredor e Organizador

Decisão	Corredor (acesso padrão) e Organizador de Grupo (permissões elevadas para gestão de grupos e eventos).
Alternativa	Três perfis (corredor / treinador / admin) ou perfil único.
Justificação	Dois perfis são suficientes para cobrir os fluxos sociais e colaborativos sem adicionar complexidade desnecessária na fase inicial.

10.3 D-03: PostgreSQL como base de dados

D-03 PostgreSQL como base de dados

Decisão	Base de dados relacional PostgreSQL com modelo normalizado.
Alternativa	MongoDB (não relacional) ou SQLite.
Justificação	O modelo de dados do RunX tem relações bem definidas (corridas, grupos, eventos, participações) que beneficiam de um esquema relacional estrito com integridade referencial. PostgreSQL é também o requisito do enunciado.

10.4 D-04: JWT para autenticação

D-04 JWT para autenticação (stateless)

Decisão	Tokens JWT com expiração de 1h + refresh tokens de 7 dias.
Alternativa	Sessions no servidor (stateful) ou cookies simples.
Justificação	JWT é stateless, adequado para uma API REST consumida por um cliente React desacoplado. Simplifica o deploy e escala horizontalmente sem necessidade de sessões partilhadas.

10.5 D-05: LLM via API externa

D-05 LLM via API externa (Anthropic/OpenAI)

Decisão	Integração com API de LLM externo para as três funcionalidades de inteligência.
Alternativa	Modelo local (Ollama) ou sem funcionalidade LLM.
Justificação	Uma API externa oferece qualidade superior sem overhead de infraestrutura. O custo é controlável com rate limiting por utilizador. A resiliência é garantida por fallbacks estáticos.

10.6 D-06: Publicações anonimizadas

D-06 Publicações anonimizadas em caso de eliminação de conta

Decisão	Ao eliminar a conta, dados pessoais são removidos mas publicações em grupos ficam anonimizadas (autor = "utilizador removido").
Alternativa	Eliminar em cascata todas as publicações do utilizador.
Justificação	A eliminação em cascata quebraria a coerência narrativa dos eventos coletivos e grupos. A anonimização respeita o RGPD sem degradar a experiência dos outros membros do grupo.

10.7 D-07: Storyboards como documento HTML

D-07 Storyboards como documento HTML interativo

Decisão	Sketches de interface desenvolvidos em HTML/CSS com alta fidelidade visual.
Alternativa	Wireframes em papel ou Figma.
Justificação	HTML permite iterar rapidamente, incluir anotações inline e partilhar sem dependência de ferramentas externas. A escolha reflete também as competências da unidade curricular.

11

Evolução do Documento para o Checkpoint 2

11.1 Objetivo da evolução

Este capítulo regista a evolução da especificação apresentada no Checkpoint 1 para a implementação do backend no Checkpoint 2, mantendo a estrutura original e explicitando o que foi concretizado, o que foi ajustado e o que permanece planeado para fases seguintes.

11.2 Reflexão por secção do documento base

11.2.1 Enquadramento

O enquadramento conceptual manteve-se estável: o foco em gestão contextual de corridas e dimensão social continua coerente com a implementação. A principal evolução foi a validação prática dos fluxos centrais através de uma API funcional com persistência em PostgreSQL.

11.2.2 Utilizadores e Cenários

Os dois perfis definidos (Runner e Organizer) foram suportados no backend com controlo de permissões em operações críticas de grupos e eventos. A reflexão principal foi a necessidade de tornar explícitas regras de autorização no serviço de domínio, e não apenas ao nível da interface.

11.2.3 Storyboards e Interação

Os storyboards mantêm-se como referência de UX, mas nesta fase serviram sobretudo como fonte de contratos de dados e navegação API. O maior ganho foi converter intenções de ecrã em endpoints concretos e respostas normalizadas.

11.2.4 Casos de Uso

Os casos de uso de autenticação, corrida, feed, grupos e metas semanais foram implementados em endpoints operacionais. Foi observado que alguns casos previstos para IA (ex.: sugestão inteligente de rota) permaneceram em modo stub, mas com contrato de endpoint já pronto para integração na fase seguinte.

11.2.5 Requisitos Funcionais

Houve avanço material em CRUD e fluxos sociais. A reflexão principal foi a importância de validação estrita dos inputs com Zod em body, params e query, para reduzir erros em runtime e alinhar com critérios de correção.

11.2.6 Requisitos Não Funcionais

A arquitetura em camadas foi aplicada (routes/controllers/services/models), com tratamento de erros centralizado e documentação Swagger interativa. A principal melhoria foi aumentar a robustez de validação e consistência de respostas para facilitar testes e manutenção.

11.2.7 Modelo de Dados e API

O modelo Prisma foi expandido para suportar entidades do domínio social e colaborativo (posts, reações, comentários, grupos, eventos, participações, notificações, metas e relatórios). A API foi alinhada com esse modelo e documentada de forma unificada no OpenAPI.

11.3 Alterações implementadas no Checkpoint 2

- Integração completa com PostgreSQL via Prisma para autenticação, corridas, metas, temas, feed, notificações e grupos.
- Endpoints de grupos e eventos coletivos com regras de autorização para organizer e membros.
- Validação de entradas com Zod aplicada a body/params/query nas rotas principais.
- Swagger expandido para todos os endpoints implementados no backend.
- Estrutura em camadas consolidada e compilação TypeScript estabilizada para entrega.

11.4 Anexo A – Estimativa de cumprimento de requisitos

11.4.1 A.1 Requisitos gerais

Categoria	Cumprimento estimado	Evidência principal
Arquitetura backend em camadas	100%	Estrutura routes/controllers/services/models ativa e usada nos fluxos principais.
Persistência de dados relacional	100%	Prisma + PostgreSQL com migrações e entidades do domínio RunX.
Validação estrita de input	95%	Zod aplicado em body/params/query nas rotas críticas; cobertura residual depende de futuros endpoints de fase 3.
Documentação de API	100%	OpenAPI/Swagger com endpoints implementados no checkpoint 2.

Tabela 11.1: Estimativa de cumprimento dos requisitos gerais

11.4.2 A.2 Requisitos específicos por fase

Fase	Cumprimento estimado	Justificação
Fase 1 (Especificação)	100%	Documento base completo e mantido como referência evolutiva.
Fase 2 (Backend)	95%	API funcional com dados persistidos, validação e documentação; integrações avançadas de IA ficam para fase seguinte.
Fase 3 (Frontend + IA)	30%	Apenas contratos e stubs preparados (ex.: sugestão de rota), sem integração final de cliente e IA.
Fase 4 (Entrega final)	10%	Fase ainda não iniciada; percentagem representa preparação documental e técnica inicial.

Tabela 11.2: Estimativa de cumprimento dos requisitos específicos por fase

11.5 Riscos remanescentes e plano imediato

- Concluir integrações de IA previstas (frases motivacionais e análises semanais) com validação de resposta.
- Garantir cobertura de testes de integração para fluxos de autorização em grupos e eventos.
- Alinhar frontend com os contratos finais da API para fecho da fase 3.

Resumo Executivo

Projeto	RunX – Plataforma Web de Gestão Diária de Corridas
Tópicos	Gestão de Tarefas Contextual e Social + Life Threads: Diário Digital Colaborativo
Utilizadores	Corredor (Runner) + Organizador de Grupo, com permissões distintas
Tecnologias	Node.js + Express - PostgreSQL - React - JWT
Serviços externos	Google Maps/Mapbox - Cloudinary/S3 - Firebase FCM - OpenWeather
LLM	Sugestão de trajetos - Frases motivacionais - Relatório semanal
Casos de uso	6 casos de uso formalizados com pré/pós-condições e fluxos alternativos
Requisitos	16 RF gerais + 21 RF por fase + 12 RNF documentados
Modelo de dados	13 entidades no diagrama E-A
API	35+ endpoints documentados + 2 fluxos de integração detalhados
Decisões	7 decisões de projeto com justificação e alternativas
Fases	F1: Especificação - F2: Backend - F3: Frontend + Integração - F4: Final
Entrega F1	18 de Março de 2026

Tabela 11.3: Resumo executivo do projeto RunX